

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Nivel 1

Ejercicio 1- Nivel 1

A partir de los documentos adjuntos (estructura_dades y dades_introduir), importa ambas tablas. Muestra las características principales del esquema creado y explica las diferentes tablas y variables que existen. Asegúrate de incluir un diagrama que ilustre la relación entre las diferentes tablas y variables.

Respuesta:

Creamos la base de datos 'transactions' y las tablas 'company' y 'transaction' con el código que contenía el archivo 'estructura_dades'.

```
-- Creamos la base de datos
• CREATE DATABASE IF NOT EXISTS transactions;
• USE transactions;

-- Creamos la tabla company
• CREATE TABLE IF NOT EXISTS company (
  id VARCHAR(15) PRIMARY KEY,
  company_name VARCHAR(255),
  phone VARCHAR(15),
  email VARCHAR(100),
  country VARCHAR(100),
  website VARCHAR(255)
);

-- Creamos la tabla transaction
• CREATE TABLE IF NOT EXISTS transaction (
  id VARCHAR(255) PRIMARY KEY,
  credit_card_id VARCHAR(15) REFERENCES credit_card(id),
  company_id VARCHAR(20),
  user_id INT REFERENCES user(id),
  lat FLOAT,
  longitude FLOAT,
  timestamp TIMESTAMP,
  amount DECIMAL(10, 2),
  declined BOOLEAN,
  FOREIGN KEY (company_id) REFERENCES company(id)
);
```

Luego introducimos los datos a ambas tablas creadas, con el código que contenía el archivo 'dades_introduir'.

Una vez importados los archivos indicados, y tras analizar la base de datos Transactions, dejo a continuación un resumen de las variables de las dos tablas que tenemos en detalle, los tipos de datos de cada variable (que se establecen claramente en el código de creación de la base de datos y tablas), así como la función de cada variable con respecto a cada tabla.

De la tabla **company**:

Variable	Tipo de dato	Función
id	texto varchar(15)	PK - Identifica de forma única con un número codificado a cada compañía
company_name	texto varchar(255)	Describe el nombre completo de cada compañía
phone	texto varchar(15)	Indica el número de teléfono (sin prefijo internacional) de cada compañía
email	texto varchar(100)	Indica la dirección electrónica de cada compañía
country	texto varchar(100)	Indica el país donde se ubica cada compañía
website	texto varchar(255)	Indica la dirección web de cada compañía

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

De la tabla **transaction**:

Variable	Tipo de dato	Función
id	texto varchar(255)	PK - Identifica de forma única cada transacción registrada
credit_card_id	texto varchar(15)	Identifica de forma única cada tarjeta de crédito
company_id	texto varchar(20)	FK- Identifica de forma única con un número codificado a cada compañía
user_id	numérico int	Identifica de forma única a cada usuario registrado para realizar transacciones y operar con una o más tarjetas de crédito
lat	numérico float	Indica la latitud desde donde se ejecuta la transacción registrada, para acceder a su ubicación geográfica precisa
longitude	numérico float	Indica la longitud desde donde se ejecuta la transacción registrada, para acceder a su ubicación geográfica precisa
timestamp	timestamp	Registra la hora precisa (hora-minutos-segundos) y la fecha (año-mes-día) en la que se ejecuta la transacción
amount	numérico decimal	Indica el monto de la transacción ejecutada (no especifica la moneda del registro)
declined	booleano	Indica si la transacción fue declinada o no, siendo 0 el indicador de "transacción no declinada" y 1 el indicador de "transacción declinada"

Posteriormente se han dividido en seis apartados, las observaciones derivadas de dicho análisis, como sigue:

1. Creación de las tablas y claves primarias

Se observa en el código la creación de dos tablas:

- **company**
- **transaction**

En ambas se define una **Primary Key (PK)** para identificar de manera única cada registro de la tabla.

- **company(id)**
- **transaction(id)**

2. Tipos de datos de texto

Las variables de texto utilizan **VARCHAR** para establecer una longitud máxima de caracteres en cada variable.

Por ejemplo:

company_name VARCHAR(255)
phone VARCHAR(15)

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

email VARCHAR(100)
country VARCHAR(100)
website VARCHAR(255)

Salta a la vista que algunas longitudes parecen muy grandes considerando el tipo de dato que se supone estamos esperando recibir, especialmente:

- **company_name VARCHAR(255)**
- **email VARCHAR(100)**
- **country VARCHAR(100)**

Sugerimos las preguntas: ¿qué determina la longitud que se establece para cada variable?, y ¿realmente es necesario permitir tantos caracteres?.

3. Relaciones mediante claves foráneas

La tabla **transaction** contiene tres variables relacionadas con otras tres tablas:

- **credit_card_id**
- **company_id**
- **user_id**

En el código aparecen referencias hacia:

- **credit_card(id)**
- **user(id)**
- **company(id)**

En los dos primeros casos, las tablas **credit_card** y **user** no aparecen creadas en el código recibido, pero la referencia permite deducir que el modelo contempla la existencia de esas tablas. En el caso de la tabla **company**, sí tenemos acceso a su contenido y detalle.

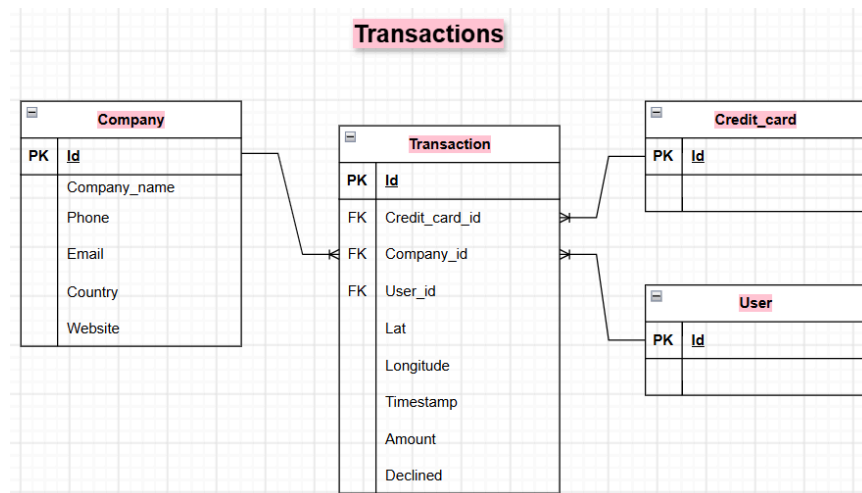
Por otro lado, el diseño establece **REFERENCES** entre la tabla **transaction** y las tablas **credit_card**, **user** y **company**. En el código proporcionado, la relación con la tabla **company** se declara explícitamente mediante una restricción **FOREIGN KEY**, mientras que las referencias a las tablas **credit_card** y **user** aparecen mediante la cláusula **REFERENCES** asociada a las columnas. Sugerimos la pregunta: ¿Podemos asumir que estas variables con **FK** que conectan con las variables **PK id** de las mencionadas tablas?

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

4. Diagrama EER

Después de analizar las tablas y variables, se crea el diagrama para mostrar las relaciones entre tablas y variables, utilizando la herramienta **draw.io**, como se muestra a continuación:



Se presentan cuatro tablas, que hasta el momento sabemos que existen dentro de esta base de datos:

- **company**
- **transaction**
- **credit_card**
- **user**

Y dentro de cada tabla se representan las variables conocidas, indicando:

- **Nombre de la variable**
- **PK o FK cuando corresponda**

Esto permite visualizar tanto la estructura de las tablas como las relaciones entre ellas, que se describen de la siguiente manera:

Company	1 ——— N Transaction
Credit_card	1 ——— N Transaction
User	1 ——— N Transaction

Interpretando que:

- Una “Company” puede estar asociada a muchas “Transactions”.
- Una “Credit_card” puede estar asociada a muchas “Transactions”.
- Un “User” puede estar asociado a muchas “Transactions”.

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

La tabla **transaction** contiene las **FK** que permiten establecer estas relaciones.

Ver referencia en punto 3.

F. Posible modelo de estrella

Podemos decir que **presenta una estructura similar a un modelo de estrella**, aunque nos haría falta tener toda la información de la base de datos y conocer su propósito para poder determinarlo.

En este supuesto, la tabla **transaction** podría considerarse la tabla de hechos, mientras que las tablas **company**, **credit_card** y **user** funcionan como tablas de dimensión.

Ejercicio 2 - Nivel 1

Hacer las siguientes consultas, usando JOIN (en base a la base de datos ya creada).

2a. Lista de países que están generando ventas.

Nota: De aquí en adelante, cuando se habla de ventas solo consideramos las transacciones exitosas o aprobadas, por lo que 'declined' debe ser igual a '0', y aplicamos este filtro en el where.

The screenshot shows a SQL IDE window with a query editor and a results pane. The query editor contains the following SQL code:

```
4 USE transactions;
5 SELECT DISTINCT company.country
6 FROM company
7 JOIN `transaction`
8 ON company.id = `transaction`.company_id
9 WHERE `transaction`.declined = 0
10 ORDER BY company.country ASC;
```

The results pane displays a list of countries: Australia, Belgium, Canada, China, France, Germany, Ireland, Italy, Netherlands, New Zealand, Norway, Spain, Sweden, United Kingdom, and United States.

Below the results pane, the 'Action Output' section shows the execution log:

#	Time	Action	Message
1	10:42:19	CREATE DATABASE IF NOT EXISTS transactions	1 row(s) affected, 1 warning(s): 1007 Can't create database 'transactions'; data
2	10:42:22	USE transactions	0 row(s) affected
3	10:42:26	SELECT DISTINCT company.country FROM company JOIN `transaction` ON company.id = `transaction`.compa...	15 row(s) returned

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

2b. A partir de cuántos países se generan las ventas.

```
-- 2b. A partir de cuántos países se generan las ventas.
14
15 • SELECT COUNT(DISTINCT company.country) AS n_paises_vendedores
16 FROM company
17 JOIN `transaction`
18 ON company.id = `transaction`.company_id
19 WHERE `transaction`.declined = 0;
20
21
```

Result Grid	
n_paises_vendedores	
15	

Result 4 x

Output

#	Time	Action	Message
1	10:42:19	CREATE DATABASE IF NOT EXISTS transactions	1 row(s) affected, 1 warning(s): 1007 Can't create database 'transactions': data
2	10:42:22	USE transactions	0 row(s) affected
3	10:42:26	SELECT DISTINCT company.country FROM company JOIN `transaction` ON company.id = `transaction`.compa...	15 row(s) returned
4	10:46:12	SELECT COUNT(DISTINCT company.country) AS n_paises_vendedores FROM company JOIN `transaction` O...	1 row(s) returned

2c. Identifica la empresa con las ventas medias más altas.

```
--
19 -- 2c. Identifica la empresa con las ventas medias más altas.
20 • SELECT company.company_name, ROUND(AVG(`transaction`.amount)) AS media_ventas
21 FROM company
22 JOIN `transaction`
23 ON company.id = `transaction`.company_id
24 WHERE `transaction`.declined = 0
25 GROUP BY company.id
26 ORDER BY media_ventas DESC
27 LIMIT 1;
28
```

company_name	media_ventas
Ac Fermentum Incorporated	285

Result 4 x

Output

#	Time	Action	Message
1	11:47:17	SELECT company.company_name, ROUND(AVG(transaction.amount)) AS media_ventas FROM company JOIN...	1 row(s) returned

Ejercicio 3 - Nivel 1

Usando solo subconsultas, sin usar JOIN, resuelve las siguientes consultas:

Nota: trabajé todos los ejercicios posteriores entendiendo que había una restricción para usar JOIN y que debía combinar tablas solo usando subconsultas. Esto por lo que se indica en la sección de OBJETIVOS al inicio de esta sección: Tarea S2.01. Conceptos básicos de SQL.

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

3a. Muestra todas las transacciones realizadas por empresas en Alemania.

Nota: cuando solicitan mostrar “todas las transacciones realizadas” entendemos que cuentan las declinadas y las exitosas, por lo cual en estos casos NO filtramos por ‘declined’ como en el caso de ventas.

```
-- Nivel 1 - Ejercicio 3
-- 3a. Muestra todas las transacciones realizadas por empresas en Alemania.
SELECT `transaction`.id, `transaction`.amount
FROM `transaction`
WHERE `transaction`.company_id IN (SELECT company.id
FROM company
WHERE company.country = 'Germany');
```

id	amount
00138D3B-206D-4C03-94B7-63A2676EB9B4	426.36
0013C1B6-3B84-4D6C-8154-E2B3FEBCA8E9	316.90
00201A11-2E62-44C4-941D-198FC80B77F0	453.04
00235618-0A5C-4D49-9DCB-83A9405D8923	263.14
005A5A7B-1F1A-4B6C-9B15-1625A78C9C38	442.01
00687139-48B2-4FFA-8E73-B20376F04AB4	524.84
0074F4DD-32F1-4827-8758-55896314623A	491.90
00AAB9CD-39D6-4DCB-8A1D-13BE73DC90A9	167.15
00BE09D4-6920-47D8-ABE8-325E2269829D	141.66
00DA0383-E048-4577-8ED1-3C56C258FF2F	325.62
00DD11DE-ED01-48BD-93A0-174D183A59D0	242.53
01449CE0-98E9-4DE5-9810-728C6BA00E6F	451.71
01758C7-241E-42DA-A8B9-9F246DBF4D2F	9.46
01ABDAB9-06E2-4CA0-A131-AEE6FF11B749	368.41
01E1C7E9A873-4A7A-A8F6-911A55N8A866	168.70

transaction 10 x

Output

#	Time	Action	Message
1	11:41:22	SELECT `transaction`.id, `transaction`.amount FROM `transaction` WHERE `transaction`.company_id IN (SELEC...	13291 row(s) returned
2	11:42:24	SELECT `transaction`.id, `transaction`.amount FROM `transaction` WHERE `transaction`.company_id IN (SELEC...	13291 row(s) returned

3b. Enumera las empresas que han realizado transacciones por un importe superior a la media de todas las transacciones.

Nota: para la media de transacciones, tomé en cuenta el monto de ‘amount’, como el monto general de la transacción realizada.

```
-- 3b. Enumera las empresas que han realizado transacciones por un importe superior a la media de todas las transacciones.
SELECT company.company_name
FROM company
WHERE company.id IN (SELECT `transaction`.company_id
FROM `transaction`
WHERE `transaction`.amount > (SELECT ROUND(AVG(`transaction`.amount)) AS media_importe_transaccion
FROM `transaction`))
ORDER BY 1;
```

company_name
A Institute
Ac Fermentum Incorporated
Ac Industries
Ac Libero Inc.
Aliquam Erat Voluptat LLP
Aliquam Iaculis Lacus Corp.
Aliquam PC
Aliquet Diam Limited
Aliquet Sem Limited

company 6 x

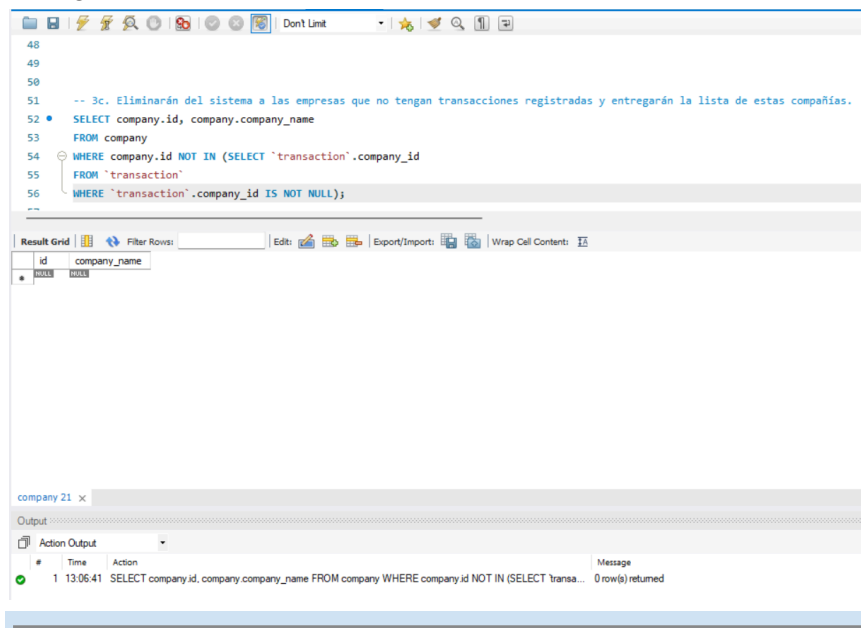
Output

#	Time	Action	Message
1	12:15:14	SELECT company.company_name FROM company WHERE company.id IN (SELECT `transaction`.company_id ...	100 row(s) returned

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

3c. Eliminarán del sistema a las empresas que no tengan transacciones registradas y entregarán la lista de estas compañías.



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, a search icon, and a 'Don't Limit' dropdown. The SQL editor contains the following code:

```
48
49
50
51 -- 3c. Eliminarán del sistema a las empresas que no tengan transacciones registradas y entregarán la lista de estas compañías.
52 • SELECT company.id, company.company_name
53 FROM company
54 WHERE company.id NOT IN (SELECT `transaction`.company_id
55 FROM `transaction`
56 WHERE `transaction`.company_id IS NOT NULL);
--
```

Below the editor is a 'Result Grid' with columns 'id' and 'company_name'. The grid is empty. At the bottom, an 'Output' window shows the execution message: '1 13:06:41 SELECT company.id, company.company_name FROM company WHERE company.id NOT IN (SELECT transa... 0 row(s) returned'.

Ejercicio 4 - Nivel 1

Tu tarea es diseñar y crear una tabla llamada "credit_card" que almacene detalles cruciales sobre las tarjetas de crédito. La nueva tabla debe ser capaz de identificar de forma única cada tarjeta y establecer una relación adecuada con las otras dos tablas ("transacción" y "compañía"). Después de crear la tabla, tendrás que introducir la información del documento llamado "dades_introduir_credit". Recuerda mostrar el diagrama y hacer una breve descripción de él.

4a. Creación de una tabla llamada 'credit_card':

Nota: cuando hice el P2P entendí que antes de crear una tabla debo correr: DROP TABLE IF EXISTS; Pero esto solo lo pude añadir en el código del archivo .sql en todos los casos donde cree tablas, mas no pude correr el código de nuevo porque supondría un problema.

Intenté ajustar los caracteres de cada VARCHAR a la naturaleza de cada dato y a lo que esperamos recibir en cada registro, sin embargo, más adelante, estandarice un poco más su definición.

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

```
53 -- Nivel 1 - Ejercicio 4
54 -- 4a. Diseñar y crear una tabla llamada "credit_card" capaz de identificar de forma única cada tarjeta
55 -- y establecer una relación adecuada con las otras dos tablas.
56 • CREATE TABLE IF NOT EXISTS credit_card (
57     id VARCHAR(15) PRIMARY KEY,
58     iban VARCHAR (34),
59     pan VARCHAR (25),
60     pin VARCHAR (10),
61     cvv VARCHAR (4),
62     expiring_date VARCHAR (15)
63 );
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

4b. Introducir la información del documento llamado "dades_introducir_creditt":

Importamos el archivo y lo ejecutamos, pero ocurre una incidencia con la columna 'expiring_date', la cual queríamos trabajar con tipo de dato DATE, pero nos dimos cuenta que los datos a insertar tenían la fecha en formato mes-día-año y no año-mes-día como lo espera recibir DATE.

Por lo tanto creamos la tabla con esta columna como texto y seguidamente vamos a actualizar el tipo de dato (TEXTO -> DATE).

```
1 -- Insertamos datos de credit_card
2 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2938', 'TR381958312213576817638661', '5424465566813633', '3257', '1')
3 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2945', 'D026854763748537475216568689', '5142423821948828', '9888', '1')
4 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2952', 'B6451VQ552718525688255', '4556 453 55 5287', '4558', '436')
5 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2959', 'C7242472244335841535', '372461377349375', '3583', '667')
6 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2966', 'B072LKTQ15627628377363', '448566 886747 7265', '4988', '1')
7 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2973', 'P787886226135892428458546', '544 5854 54543 384', '8788')
8 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2980', 'DE39241881883886277136', '482488 7145845669', '5075', '51')
9 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2987', 'GE89681434837748781813', '3763 747687 76666', '2298', '75')
10 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2994', 'B802714428368868705294', '344383273252593', '7545', '505')
11 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3001', 'CV49887426554754581266832118', '511722 924833 2244', '1954')
12 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3008', 'LUS87216693616119238', '4485744464433884', '1856', '748')
13 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3015', 'P5119398216295715968342456821', '3784 662233 17389', '329')
14 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3022', 'G78189516285856097742131897', '5164 1379 4862 3951', '16')
15 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3029', 'A262317413982441418123739746', '3429 279566 77631', '9788')
16 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3036', 'AZ39336882925842805843941994', '3768 451556 48766', '2231')
17 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3043', 'T86488143318514852179535', '455676 643746365', '5969', '1')
18 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3050', 'FR516774436917583681854477', '4824887123722', '4834', '1')
19 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3057', 'L9331822574697545215', '3484 621767 21237', '6885', '648')
20 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3064', 'P514688554548925337822773157', '3467 732741 26818', '388')
21 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3071', 'N08923814763512', '3464 789562 23352', '6625', '661', '11')
22 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3078', '15825127145884623279548733', '4539 322 74 2377', '9485', '1')
23 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3085', 'B603114723972437', '5266 3346 1135 1687', '7241', '413')
24 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3092', 'B06515001165122125447487', '3488 754223 46253', '9412', '1')
25 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3099', 'D07458763683706873318', '448 6618 8865 788', '1671')
26 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3106', 'D07458763683706873318', '448 6618 8865 788', '1671')
27 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3113', 'D07458763683706873318', '448 6618 8865 788', '1671')
28 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3120', 'D07458763683706873318', '448 6618 8865 788', '1671')
29 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3127', 'D07458763683706873318', '448 6618 8865 788', '1671')
30 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3134', 'D07458763683706873318', '448 6618 8865 788', '1671')
31 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3141', 'D07458763683706873318', '448 6618 8865 788', '1671')
32 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3148', 'D07458763683706873318', '448 6618 8865 788', '1671')
33 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3155', 'D07458763683706873318', '448 6618 8865 788', '1671')
34 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3162', 'D07458763683706873318', '448 6618 8865 788', '1671')
35 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3169', 'D07458763683706873318', '448 6618 8865 788', '1671')
36 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3176', 'D07458763683706873318', '448 6618 8865 788', '1671')
37 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3183', 'D07458763683706873318', '448 6618 8865 788', '1671')
38 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3190', 'D07458763683706873318', '448 6618 8865 788', '1671')
39 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3197', 'D07458763683706873318', '448 6618 8865 788', '1671')
40 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3204', 'D07458763683706873318', '448 6618 8865 788', '1671')
41 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3211', 'D07458763683706873318', '448 6618 8865 788', '1671')
42 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3218', 'D07458763683706873318', '448 6618 8865 788', '1671')
43 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3225', 'D07458763683706873318', '448 6618 8865 788', '1671')
44 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3232', 'D07458763683706873318', '448 6618 8865 788', '1671')
45 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3239', 'D07458763683706873318', '448 6618 8865 788', '1671')
46 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3246', 'D07458763683706873318', '448 6618 8865 788', '1671')
47 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3253', 'D07458763683706873318', '448 6618 8865 788', '1671')
48 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3260', 'D07458763683706873318', '448 6618 8865 788', '1671')
49 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3267', 'D07458763683706873318', '448 6618 8865 788', '1671')
50 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3274', 'D07458763683706873318', '448 6618 8865 788', '1671')
51 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3281', 'D07458763683706873318', '448 6618 8865 788', '1671')
52 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3288', 'D07458763683706873318', '448 6618 8865 788', '1671')
53 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3295', 'D07458763683706873318', '448 6618 8865 788', '1671')
54 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3302', 'D07458763683706873318', '448 6618 8865 788', '1671')
55 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3309', 'D07458763683706873318', '448 6618 8865 788', '1671')
56 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3316', 'D07458763683706873318', '448 6618 8865 788', '1671')
57 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3323', 'D07458763683706873318', '448 6618 8865 788', '1671')
58 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3330', 'D07458763683706873318', '448 6618 8865 788', '1671')
59 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3337', 'D07458763683706873318', '448 6618 8865 788', '1671')
60 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3344', 'D07458763683706873318', '448 6618 8865 788', '1671')
61 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3351', 'D07458763683706873318', '448 6618 8865 788', '1671')
62 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3358', 'D07458763683706873318', '448 6618 8865 788', '1671')
63 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3365', 'D07458763683706873318', '448 6618 8865 788', '1671')
64 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3372', 'D07458763683706873318', '448 6618 8865 788', '1671')
65 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3379', 'D07458763683706873318', '448 6618 8865 788', '1671')
66 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3386', 'D07458763683706873318', '448 6618 8865 788', '1671')
67 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3393', 'D07458763683706873318', '448 6618 8865 788', '1671')
68 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3400', 'D07458763683706873318', '448 6618 8865 788', '1671')
69 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3407', 'D07458763683706873318', '448 6618 8865 788', '1671')
70 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3414', 'D07458763683706873318', '448 6618 8865 788', '1671')
71 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3421', 'D07458763683706873318', '448 6618 8865 788', '1671')
72 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3428', 'D07458763683706873318', '448 6618 8865 788', '1671')
73 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3435', 'D07458763683706873318', '448 6618 8865 788', '1671')
74 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3442', 'D07458763683706873318', '448 6618 8865 788', '1671')
75 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3449', 'D07458763683706873318', '448 6618 8865 788', '1671')
76 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3456', 'D07458763683706873318', '448 6618 8865 788', '1671')
77 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3463', 'D07458763683706873318', '448 6618 8865 788', '1671')
78 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3470', 'D07458763683706873318', '448 6618 8865 788', '1671')
79 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3477', 'D07458763683706873318', '448 6618 8865 788', '1671')
80 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3484', 'D07458763683706873318', '448 6618 8865 788', '1671')
81 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3491', 'D07458763683706873318', '448 6618 8865 788', '1671')
82 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3498', 'D07458763683706873318', '448 6618 8865 788', '1671')
83 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3505', 'D07458763683706873318', '448 6618 8865 788', '1671')
84 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3512', 'D07458763683706873318', '448 6618 8865 788', '1671')
85 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3519', 'D07458763683706873318', '448 6618 8865 788', '1671')
86 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3526', 'D07458763683706873318', '448 6618 8865 788', '1671')
87 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3533', 'D07458763683706873318', '448 6618 8865 788', '1671')
88 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3540', 'D07458763683706873318', '448 6618 8865 788', '1671')
89 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3547', 'D07458763683706873318', '448 6618 8865 788', '1671')
90 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3554', 'D07458763683706873318', '448 6618 8865 788', '1671')
91 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3561', 'D07458763683706873318', '448 6618 8865 788', '1671')
92 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3568', 'D07458763683706873318', '448 6618 8865 788', '1671')
93 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3575', 'D07458763683706873318', '448 6618 8865 788', '1671')
94 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3582', 'D07458763683706873318', '448 6618 8865 788', '1671')
95 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3589', 'D07458763683706873318', '448 6618 8865 788', '1671')
96 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3596', 'D07458763683706873318', '448 6618 8865 788', '1671')
97 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3603', 'D07458763683706873318', '448 6618 8865 788', '1671')
98 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3610', 'D07458763683706873318', '448 6618 8865 788', '1671')
99 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3617', 'D07458763683706873318', '448 6618 8865 788', '1671')
100 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3624', 'D07458763683706873318', '448 6618 8865 788', '1671')
101 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3631', 'D07458
```

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

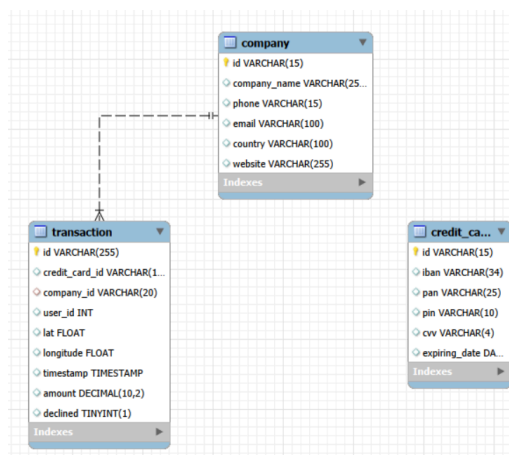
Finalmente, cambiamos el tipo de dato de VARCHAR a DATE

```
64 -- 4b. Introducir la información del documento llamado "dades_introduir_credit".
65 -- Se ejecutó el archivo y se introdujeron así los datos insertados en la tabla 'credit_card'
66 -- Ahora actualizamos la tabla, reescribiendo la interpretación de los datos como mes-día-año
67 -- Pero antes necesitaremos desactivar temporalmente el modo seguro de SQL porque SI requerimos afectar todas las filas en este UPDATE
68 • SET SQL_SAFE_UPDATES = 0;
69 • UPDATE credit_card
70   SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y');
71 -- Reactivamos el modo seguro por buenas prácticas
72 • SET SQL_SAFE_UPDATES = 1;
73 -- Ahora si cambiamos el tipo de dato de VARCHAR a DATE
74 • ALTER TABLE credit_card
75   MODIFY COLUMN expiring_date DATE;
```

Output			
Action Output			
#	Time	Action	Message
1	21:42:19	SET SQL_SAFE_UPDATES = 0	0 row(s) affected
2	21:47:05	UPDATE credit_card SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y')	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0
3	21:48:26	SET SQL_SAFE_UPDATES = 1	0 row(s) affected
4	21:56:11	ALTER TABLE credit_card MODIFY COLUMN expiring_date DATE	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

4c. Mostrar el diagrama y hacer una breve descripción del mismo.

En una primera visualización del diagrama EER en MYSQL Workbench, observamos que la tabla 'credit_card' no está relacionada a ninguna otra de esta base de datos.



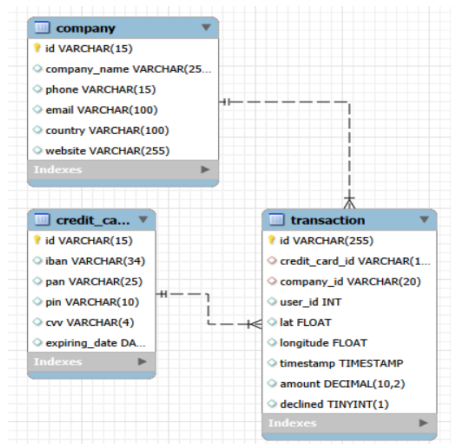
Por lo tanto, decidimos modificar la tabla 'transaction', añadiendo una FK que la vincule a la tabla 'credit_card' que acabamos de crear, ya que la tabla 'transaction' originalmente solo tiene una FK creada que la vincula a la tabla 'company'.

```
77 -- 4c. Visualización del diagrama de la BDD 'transactions'
78 -- En una primera visualización vemos que la tabla 'credit_card' no esta vinculada a ninguna otra
79 -- Creamos una modificación en la tabla 'transaction', añadiendo una foreign key que la vincule a la tabla 'credit_card'
80 • ALTER TABLE 'transaction'
81   ADD FOREIGN KEY (credit_card_id) REFERENCES credit_card(id);
82
```

Output			
Action Output			
#	Time	Action	Message
1	21:42:19	SET SQL_SAFE_UPDATES = 0	0 row(s) affected
2	21:47:05	UPDATE credit_card SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y')	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0
3	21:48:26	SET SQL_SAFE_UPDATES = 1	0 row(s) affected
4	21:56:11	ALTER TABLE credit_card MODIFY COLUMN expiring_date DATE	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
5	22:24:46	ALTER TABLE 'transaction' ADD FOREIGN KEY (credit_card_id) REFERENCES credit_card(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón



Ejercicio 5 - Nivel 1

El departamento de Recursos Humanos ha identificado un error en el número de cuenta asociado a la tarjeta de crédito con el ID CcU-2938. La información que debe mostrarse para este registro es: TR323456312213576817699999. Recuerda mostrar que el cambio se ha hecho.

Respuesta:

Lo primero es que con UPDATE, modificamos el registro IBAN asociado a la PK ID, donde ID = CcU-2938.

```
83 -- Nivel 1 - Ejercicio 5
84 -- Ejercicio 5
85 -- El departamento de Recursos Humanos ha identificado un error en el número de cuenta asociado a la tarjeta de crédito
86 -- con el ID CcU-2938. La información que debe mostrarse para este registro es: TR323456312213576817699999.
87 UPDATE credit_card
88 SET iban = 'TR323456312213576817699999'
89 WHERE id = 'CcU-2938';
90
91
92
```

Output		
Action Output		
#	Time	Action
1	10.33.21	UPDATE credit_card SET iban = 'TR323456312213576817699999' WHERE id = 'CcU-2938'
		Message
		1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

Ahora comprobamos que la fila ha quedado con la información correcta.

```
91 -- Ahora mostramos como quedó esa fila
92 SELECT credit_card.id, credit_card.iban
93 FROM credit_card
94 WHERE credit_card.id = 'CcU-2938';
95
```

Result Grid		
id	iban	
CcU-2938	TR323456312213576817699999	

credit_card 2 x		
Output		
Action Output		
#	Time	Action
1	10.33.21	UPDATE credit_card SET iban = 'TR323456312213576817699999' WHERE id = 'CcU-2938'
		Message
		1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
2	10.36.50	SELECT credit_card.id, credit_card.iban FROM credit_card WHERE credit_card.id = 'CcU-2938'
		Message
		1 row(s) returned

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Ejercicio 6 - Nivel 1

En la tabla de "transacciones", introduzca una nueva transacción con la siguiente información:

Id = 108B1D1D-5B23-A76C-55EF-C568E49A99DD

credit_card_id = CcU-9999 // **company_id** = b-9999 // **user_id** = 9999

lat = 829.999 // **longitude** = -117.999 // **amount** = 111.11 // **declined** = 0

Primero insertamos el dato en la tabla 'company', en su columna 'id', ya que no podemos crear un registro de un nuevo company_id en la tabla transaction sin que antes esté registrado ese id en la tabla company, considerando que en la tabla 'transaction' tenemos que 'company_id' es una FK y hace REFERENCES a la tabla 'company' a través de su PK 'id', y para eso nos sirve la restricción de la referencia, para cuidarnos de ese error.

Validamos que la tabla company admite valores nulos en las columnas restantes a través de DESCRIBE, por lo que no habrá problema, ya que solo tenemos el dato id para este nuevo registro.

```
96 -- Nivel 1 - Ejercicio 6
97 -- En la tabla de "transacciones", introduzca una nueva transacción con la siguiente información:
98 -- Id -> 108B1D1D-5B23-A76C-55EF-C568E49A99DD
99 -- credit_card_id -> CcU-9999 // company_id -> b-9999
100 -- user_id -> 9999 // lat -> 829.999
101 -- longitude -> -117.999 // amount -> 111.11
102 -- declined -> 0
103 • INSERT INTO company (id) VALUES ('b-9999');
104
105
106
107
```

#	Time	Action	Message
1	11:15:05	INSERT INTO company (id) VALUES ('b-9999')	1 row(s) affected

Verificamos que se ha registrado efectivamente este nuevo dato.

```
103 • INSERT INTO company (id) VALUES ('b-9999');
104 • SELECT company.id
105 FROM company
106 WHERE company.id = 'b-9999';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Contents: |

id
b-9999

company 6 x

#	Time	Action	Message
1	11:15:05	INSERT INTO company (id) VALUES ('b-9999')	1 row(s) affected
2	11:18:56	SELECT company.id FROM company WHERE company.id = 'b-9999'	1 row(s) returned

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Lo mismo debe hacerse con la tabla `credit_card`, e insertar el registro de `id` previo a insertar el registro en la tabla `transaction`, porque en ejercicios anteriores añadimos una nueva FK a la tabla `transaction` (`credit_card_id`).

```
110 -- Insertamos ahora el nuevo id en la tabla credit_card (Tabla padre, por referencia)
111 • INSERT INTO credit_card (id) VALUES ('CcU-9999');
112
113 -- Ahora procedemos con el registro en tabla transaction
```

#	Time	Action	Message
1	11:22:52	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) ...	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (t...
2	11:23:17	INSERT INTO company (id) VALUES (b-9999)	Error Code: 1062. Duplicate entry 'b-9999' for key 'company.PRIMARY'
3	11:23:26	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) ...	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (t...
4	11:26:36	DESCRIBE credit_card	6 row(s) returned
5	11:27:24	INSERT INTO credit_card (id) VALUES (CcU-9999)	1 row(s) affected

Verificamos que se ha registrado efectivamente.

```
112 -- Validamos que se registró correctamente
113 • SELECT credit_card.id
114 FROM credit_card
115 WHERE credit_card.id = 'CcU-9999';
116
```

#	Time	Action	Message
1	11:31:40	SELECT credit_card.id FROM credit_card WHERE credit_card.id = 'CcU-9999'	1 row(s) returned

Finalmente procedemos con el registro en la tabla 'transaction'.

```
116 -- Ahora procedemos con el registro en tabla transaction
117 • INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('108B1D1D-5B23-A76C-55E
118
```

#	Time	Action	Message
1	11:39:42	INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) V...	1 row(s) affected

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Validamos que el registro se haya realizado correctamente.

```
116 -- Ahora procedemos con el registro en tabla transaction
117 • INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('10881D1D-5B23-A':
118 -- Validamos que se registró correctamente
119 • SELECT *
120 FROM `transaction`
121 WHERE `transaction`.credit_card_id = 'CcU-9999';
```

The screenshot shows a database client interface. At the top, there's a toolbar with icons for various functions. Below it, a 'Result Grid' displays a single row of data from a query. The columns are: id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, and declined. The data row shows: 10881D1D-5B23-A76C-55EF-C568E49A99CD, CcU-9999, b-9999, 9999, 829.999, -117.999, 111.11, 0. Below the result grid, there's an 'Output' window showing the execution of the query. It includes a table with columns: #, Time, Action, and Message. The message indicates that 1 row(s) were returned.

#	Time	Action	Message
1	11:50:39	SELECT * FROM `transaction` WHERE `transaction`.credit_card_id = 'CcU-9999'	1 row(s) returned

Ejercicio 7 - Nivel 1

Recursos Humanos te pide que elimines la columna "pan" de la tabla credit_card. Recuerda mostrar el cambio realizado.

Respuesta:

Para iniciar vamos a visualizar la tabla credit_card.

```
123 -- Nivel 1 - Ejercicio 7
124 -- Elimina la columna "pan" de la tabla credit_card. Recuerda mostrar el cambio realizado.
125 • DESCRIBE credit_card;
```

The screenshot shows a database client interface. At the top, there's a toolbar with icons for various functions. Below it, a 'Result Grid' displays the result of a DESCRIBE query. The columns are: Field, Type, Null, Key, Default, and Extra. The data rows show the structure of the credit_card table: id (varchar(15), NO, PRI, NULL), iban (varchar(34), YES, NULL), pan (varchar(25), YES, NULL), pin (varchar(10), YES, NULL), cvv (varchar(4), YES, NULL), and expiring_date (date, YES, NULL). Below the result grid, there's an 'Output' window showing the execution of the query. It includes a table with columns: #, Time, Action, and Message. The message indicates that 6 row(s) were returned.

Field	Type	Null	Key	Default	Extra
id	varchar(15)	NO	PRI	NULL	
iban	varchar(34)	YES		NULL	
pan	varchar(25)	YES		NULL	
pin	varchar(10)	YES		NULL	
cvv	varchar(4)	YES		NULL	
expiring_date	date	YES		NULL	

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

También verificamos con SHOW CREATE TABLE si hay alguna restricción relacionada a esta columna, como buena práctica para evitar afectar otras tablas con este cambio.

```
123 -- Nivel 1 - Ejercicio 7
124 -- Elimina la columna "pan" de la tabla credit_card. Recuerda mostrar el cambio realizado.
125 • DESCRIBE credit_card;
126 • SHOW CREATE TABLE credit_card;
127
```

Table	Create Table
credit_card	CREATE TABLE `credit_card` (`id` varchar(15) NOT NULL, `iban` varchar(34) DEFAULT NULL, `pan` varchar(25) DEFAULT NULL, `pi...

#	Time	Action	Message
1	12:22:47	DESCRIBE credit_card	6 row(s) returned
2	12:27:02	SHOW CREATE TABLE credit_card	1 row(s) returned

Eliminamos la columna pan de la tabla credit_card.

```
123 -- Nivel 1 - Ejercicio 7
124 -- Elimina la columna "pan" de la tabla credit_card. Recuerda mostrar el cambio realizado.
125 -- Miramos el contenido de la tabla
126 • DESCRIBE credit_card;
127 -- Revisamos como fue creada la tabla para entender si hay restricciones o relaciones que sean afectadas por este comando
128 • SHOW CREATE TABLE credit_card;
129 -- Ahora eliminamos la columna
130 • ALTER TABLE credit_card
131 DROP COLUMN pan;
132
```

#	Time	Action	Message
1	12:22:47	DESCRIBE credit_card	6 row(s) returned
2	12:27:02	SHOW CREATE TABLE credit_card	1 row(s) returned
3	12:35:57	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

Volvemos a mirar la tabla con DESCRIBE para validar que se haya eliminado la columna pan correctamente.

```
132 -- Ahora miramos nuevamente la tabla para verificar que se elimino la columna
133 • DESCRIBE credit_card;
---
```

Field	Type	Null	Key	Default	Extra
id	varchar(15)	NO	PRI		
iban	varchar(34)	YES			
pin	varchar(10)	YES			
cvv	varchar(4)	YES			
expiring_date	date	YES			

#	Time	Action	Message
1	12:22:47	DESCRIBE credit_card	6 row(s) returned
2	12:27:02	SHOW CREATE TABLE credit_card	1 row(s) returned
3	12:35:57	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
4	12:37:18	DESCRIBE credit_card	5 row(s) returned

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Ejercicio 8 - Nivel 1

Descarga los archivos CSV que encontrarás en la sección de recursos:

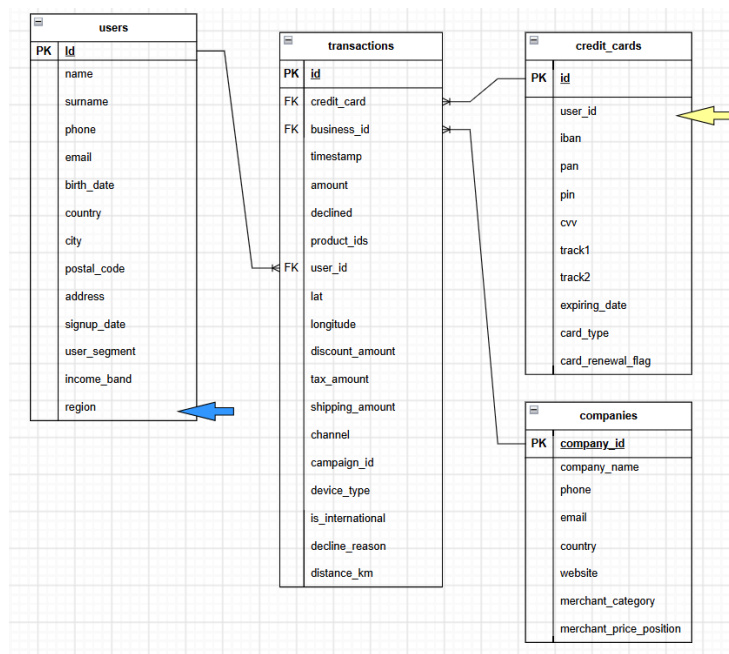
american_users.csv // european_users.csv // companies.csv // credit_cards.csv // transactions.csv

Estúdialos, y diseña una base de datos con un esquema de estrellas que contenga al menos 4 tablas, de las cuales puedes hacer las siguientes consultas:

La tabla products.csv la usaremos más adelante.

Respuesta:

Lo primero que hicimos fue dibujar el modelo en draw.io, para entender cómo relacionarlo y de ahí plantearnos un paso a paso para resolver este ejercicio.



Pudimos determinar que era necesario unificar las tablas american_users y european_users, por lo que crearemos una tabla única para users (tabla intermedia), agregando una nueva columna llamada región (ver flecha azul en el dibujo) donde indicaremos si el usuario es europeo o americano.

Además consideramos si es necesario eliminar la columna user_id de la tabla credit_cards (ver flecha amarilla en el dibujo) para eliminar la relación de esta tabla con la tabla users y así cumplir el objetivo de diseñar la base de datos como un modelo de estrella de al menos 4 tablas. Sin embargo, viendo el caso desde un punto de vista de negocios y no solo de un ejercicio de estudio, encuentro poco conveniente que la tabla de tarjetas de crédito dependa únicamente de la tabla transacciones para ubicar al usuario titular del producto, ya que si no se realizará ninguna transacción con dicha tarjeta no tendríamos acceso a esta información posteriormente.

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

El proceso implica los siguientes pasos a seguir en orden:

Paso 1. Crear la base de datos.

Paso 2. Crear una a una las tablas hijas (tablas de dimensiones) y cargar la data de cada una.

Paso 3. Ajustar cada tabla individualmente.

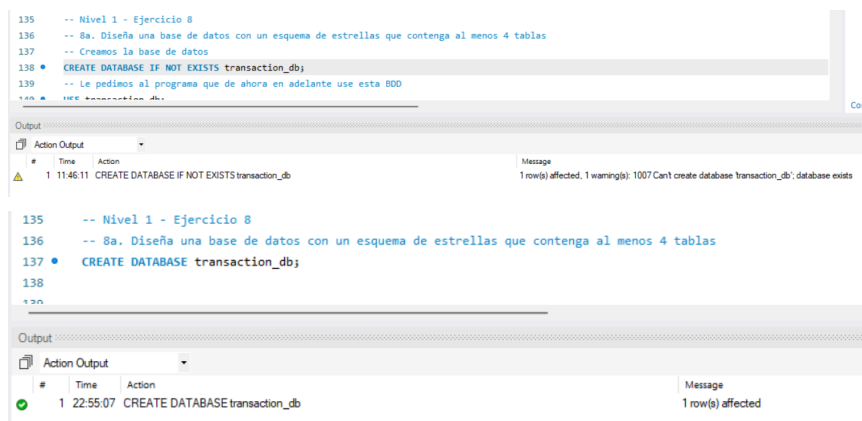
(en cuanto a tipos de datos, para que puedan ser funcionales a las consultas futuras por su naturaleza).

Paso 4. Crear la tabla de hechos, determinando PK y FK, y estableciendo relaciones con las tablas de dimensiones.

Paso 1. Crear la base de datos.

Creamos la base de datos que denominaremos: 'transaction_db'.

Nota: primero lo había creado sin el IF NOT EXISTS, por eso al corregirlo y ejecutar dice que ya existe.



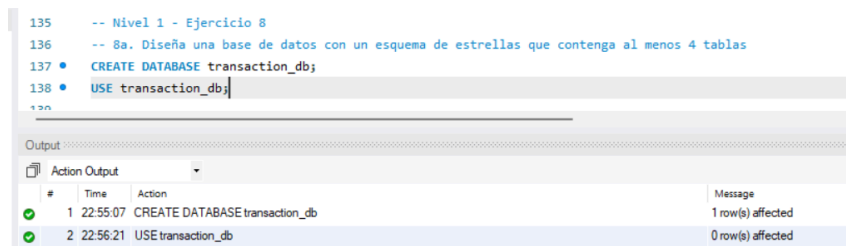
```
135 -- Nivel 1 - Ejercicio 8
136 -- 8a. Diseña una base de datos con un esquema de estrellas que contenga al menos 4 tablas
137 -- Creamos la base de datos
138 • CREATE DATABASE IF NOT EXISTS transaction_db;
139 -- Le pedimos al programa que de ahora en adelante use esta BDD
140 • USE transaction_db;
```

#	Time	Action	Message
1	11:46:11	CREATE DATABASE IF NOT EXISTS transaction_db	1 row(s) affected, 1 warning(s): 1007 Can't create database 'transaction_db': database exists

```
135 -- Nivel 1 - Ejercicio 8
136 -- 8a. Diseña una base de datos con un esquema de estrellas que contenga al menos 4 tablas
137 • CREATE DATABASE transaction_db;
138
139
```

#	Time	Action	Message
1	22:55:07	CREATE DATABASE transaction_db	1 row(s) affected

Ahora le pedimos al programa que de aquí en adelante use esta base de datos que acabamos de crear:



```
135 -- Nivel 1 - Ejercicio 8
136 -- 8a. Diseña una base de datos con un esquema de estrellas que contenga al menos 4 tablas
137 • CREATE DATABASE transaction_db;
138 • USE transaction_db;
```

#	Time	Action	Message
1	22:55:07	CREATE DATABASE transaction_db	1 row(s) affected
2	22:56:21	USE transaction_db	0 row(s) affected

Paso 2. Crear una a una las tablas hijas (tablas de dimensiones) y cargar la data de cada una.

Nota: en este punto creamos las tablas 'american_users' y 'european_users' con el objetivo poder comparar más adelante si existe algún id duplicado entre ambas. Luego de esta verificación podremos eliminar estas tablas y quedarnos con una tabla puente unificada que llamaremos 'users'.

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Crearemos la tabla `american_users`:

```
143 -- Creamos la tabla american_users
144 CREATE TABLE IF NOT EXISTS american_users(
145     id INT AUTO_INCREMENT PRIMARY KEY,
146     name VARCHAR(25),
147     surname VARCHAR(25),
148     phone VARCHAR(20),
149     email VARCHAR(25),
150     birth_date VARCHAR(20),
151     country VARCHAR(20),
152     city VARCHAR(20),
153     postal_code VARCHAR(10),
154     address VARCHAR(50),
155     signup_date DATE,
156     user_segment VARCHAR(25),
157     income_band VARCHAR(15));
158
159 -- cargamos la data para american_users desde el archivo csv
```

Output			
Action Output			
#	Time	Action	Message
1	11:46:11	CREATE DATABASE IF NOT EXISTS transaction_db	1 row(s) affected, 1
2	12:05:58	CREATE TABLE IF NOT EXISTS american_users(id INT AUTO_INCREMENT PRIMARY KEY, name VARCHA...	0 row(s) affected

Verificamos cual es la carpeta segura de MYSQL para guardar ahí nuestros archivos csv y poder importarlos a las tablas.

```
159 SHOW VARIABLES LIKE 'secure_file_priv';
```

Variable_name	Value
secure_file_priv	C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\

Result 6 x			
Output			
Action Output			
#	Time	Action	Message
1	12:54:26	CREATE TABLE IF NOT EXISTS american_users(id INT AUTO_INCREMENT PRIMARY KEY, name VARCHA...	0 row(s) affected
2	12:54:44	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned

Cargamos la data para la tabla `american_users` desde el archivo csv:

```
161 -- cargamos la data para american_users desde el archivo csv
162 LOAD DATA INFILE
163 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__american_users.csv'
164 INTO TABLE american_users
165 FIELDS TERMINATED BY ','
166 ENCLOSED BY ''
167 IGNORE 1 ROWS;
168
169 -- cargamos la data para american_users desde el archivo csv
```

Output			
Action Output			
#	Time	Action	Message
1	12:54:26	CREATE TABLE IF NOT EXISTS american_users(id INT AUTO_INCREMENT PRIMARY KEY, name VARCHA...	0 row(s) affected
2	12:54:44	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned
3	12:57:35	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__american_users.csv' INT...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Creamos la tabla `american_users`:

```
169 -- creamos la tabla american_users
170 CREATE TABLE IF NOT EXISTS american_users(
171   id INT AUTO_INCREMENT PRIMARY KEY,
172   name VARCHAR(25),
173   surname VARCHAR(25),
174   phone VARCHAR(25),
175   email VARCHAR(50),
176   birth_date VARCHAR(25),
177   country VARCHAR(25),
178   city VARCHAR(25),
179   postal_code VARCHAR(15),
180   address VARCHAR(50),
181   signup_date DATE,
182   user_segment VARCHAR(25),
183   income_band VARCHAR(15));
```

Output

#	Time	Action	Message
1	12:54:26	CREATE TABLE IF NOT EXISTS american_users(id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(25), surname VARCHAR(25), phone VARCHAR(25), email VARCHAR(50), birth_date VARCHAR(25), country VARCHAR(25), city VARCHAR(25), postal_code VARCHAR(15), address VARCHAR(50), signup_date DATE, user_segment VARCHAR(25), income_band VARCHAR(15));	0 row(s) affected
2	12:54:44	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned
3	12:57:35	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8_american_users.csv' INTO TABLE american_users FIELDS TERMINATED BY ',' ENCLOSED BY '"' IGNORE 1 ROWS;	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0
4	13:00:08	CREATE TABLE IF NOT EXISTS european_users(id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(25), surname VARCHAR(25), phone VARCHAR(25), email VARCHAR(50), birth_date VARCHAR(25), country VARCHAR(25), city VARCHAR(25), postal_code VARCHAR(15), address VARCHAR(50), signup_date DATE, user_segment VARCHAR(25), income_band VARCHAR(15));	0 row(s) affected

Cargamos la data para la tabla `european_users` desde el archivo csv:

```
185 -- cargamos la data para european_users desde el archivo csv
186 LOAD DATA INFILE
187 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8_european_users.csv'
188 INTO TABLE european_users
189 FIELDS TERMINATED BY ','
190 ENCLOSED BY '"'
191 IGNORE 1 ROWS;
```

Output

#	Time	Action	Message
1	11:07:20	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8_european_users.csv' INTO TABLE european_users FIELDS TERMINATED BY ',' ENCLOSED BY '"' IGNORE 1 ROWS;	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0

Verificamos si existen id duplicados comparando las tablas `american_users` y `european_users` y validamos que NO, por lo que podemos seguir adelante y crear una tabla `users` cargando simplemente la información de estas dos tablas en ella.

```
194 -- verificamos si existen id duplicados comparando las tablas american_users y european_users
195 SELECT id FROM american_users
196 WHERE id IN (SELECT id FROM european_users);
```

Result Grid

id
1000

american_users 21 x

Output

#	Time	Action	Message
1	11:07:20	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8_european_users.csv' INTO TABLE european_users FIELDS TERMINATED BY ',' ENCLOSED BY '"' IGNORE 1 ROWS;	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0
2	11:08:09	SELECT id FROM american_users WHERE id IN (SELECT id FROM european_users);	0 row(s) returned

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Creamos una tabla intermedia que llamamos users, donde unificamos las tablas european_users y american_users, esto con el objetivo de cumplir con el modelo de estrella requerido para esta BDD y para optimizarla, ya que esta tabla contiene la misma estructura de las dos anteriores, pero añadimos una columna llamada 'region' para poder diferenciar a los usuarios europeos y los americanos.

```
197 -- creamos una tabla intermedia que llamamos users, donde unificaremos european_users y american_users
198 • CREATE TABLE IF NOT EXISTS users(
199   id INT AUTO_INCREMENT PRIMARY KEY,
200   name VARCHAR(25),
201   surname VARCHAR(25),
202   phone VARCHAR(25),
203   email VARCHAR(50),
204   birth_date VARCHAR(25),
205   country VARCHAR(25),
206   city VARCHAR(25),
207   postal_code VARCHAR(15),
208   address VARCHAR(50),
209   signup_date DATE,
210   user_segment VARCHAR(25),
211   income_band VARCHAR(15),
212   region VARCHAR(15));
```

#	Time	Action	Message
1	13:08:46	CREATE TABLE IF NOT EXISTS users(id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(25), surname...	0 row(s) affected

Insertamos la data de american_users en la nueva tabla users, agregando el dato 'America' en la columna región.

```
214 -- insertamos la data de american_users en la tabla users, agregando el dato 'America' en la columna region
215 • INSERT INTO users (id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band, region)
216   SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band, 'America'
217   FROM american_users;
```

#	Time	Action	Message
1	13:08:46	CREATE TABLE IF NOT EXISTS users(id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(25), surname...	0 row(s) affected
2	13:12:20	INSERT INTO users (id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band, region) SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band, 'America' FROM american_users;	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

Repetimos el proceso con la data de la tabla european_users, esta vez agregando el dato 'Europa' en la columna región.

```
219 -- insertamos la data de european_users en la tabla users, agregando el dato 'Europa' en la columna region
220 • INSERT INTO users (id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band, region)
221   SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band, 'Europa'
222   FROM european_users;
```

#	Time	Action	Message
1	13:08:46	CREATE TABLE IF NOT EXISTS users(id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(25), surname...	0 row(s) affected
2	13:12:20	INSERT INTO users (id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band, region) SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band, 'America' FROM american_users;	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
3	13:13:31	INSERT INTO users (id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band, region) SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band, 'Europa' FROM european_users;	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

Spring 2. Conceptos básicos de SQL

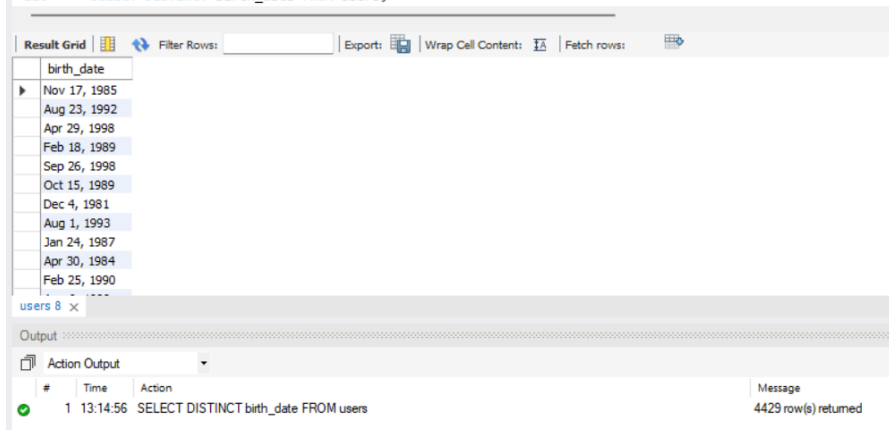
Realizado por: Lorena Morón

Paso 3. Ajustar cada tabla individualmente.

(en cuanto a tipos de datos, para que puedan ser funcionales a las consultas futuras por su naturaleza).

Ahora nuestro objetivo es trabajar en las modificaciones necesarias en la tabla users, en este caso sería modificar el tipo de dato de la columna birth_date de VARCHAR a DATE. Para ello, primero verificamos que se cumpla el mismo patrón en esta columna, chequeando los DISTINCT.

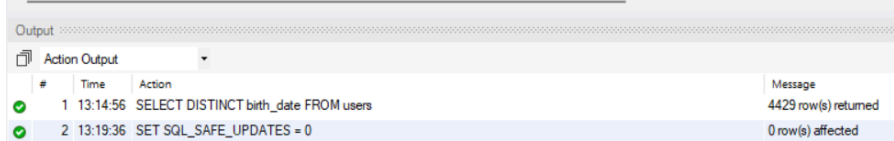
```
224 -- queremos modificar el tipo de dato de birth_date de VARCHAR a DATE para que este dato sea mas util
225 -- primero verificamos que todas las filas tengan el mismo patron en el dato
226 • SELECT DISTINCT birth_date FROM users;
```



Como efectivamente se cumple un patrón, podemos modificar todos los registros de la columna con un solo código.

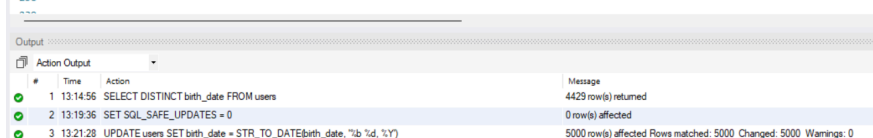
Desactivaremos el modo seguro SQL, ya que vamos a correr un código que afectará toda las filas de la tabla.

```
229 -- desactivamos el modo seguro SQL ya que vamos a afectar todas las filas
230 • SET SQL_SAFE_UPDATES = 0;
231
232
233
```



Finalmente, modificaremos el formato a través de un UPDATE de la columna birth_date atendiendo a su diseño específico, en este caso %b representa el mes escrito en letras, se deja un espacio que está presente entre esto y el número que indica el día, que sería %d, y finalmente %y representa el número que indica el año.

```
235 -- modificamos el formato
236 • UPDATE users
237 SET birth_date = STR_TO_DATE(birth_date, '%b %d, %Y');
238
```



Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Activamos nuevamente el modo seguro de MYSQL por buenas prácticas:

```
237 -- activamos nuevamente el modo seguro SQL
238 • SET SQL_SAFE_UPDATES = 1;
```

```
239
```

Output

Action Output

#	Time	Action	Message
1	13:24:32	SET SQL_SAFE_UPDATES = 1	0 row(s) affected

Cambiamos el tipo de dato de la columna:

```
238 -- Ahora cambiamos el tipo de columna a DATE
```

```
239 • ALTER TABLE users
```

```
240 MODIFY COLUMN birth_date DATE;
```

```
241
```

```
242
```

Output

Action Output

#	Time	Action	Message
1	13:24:32	SET SQL_SAFE_UPDATES = 1	0 row(s) affected
2	13:26:23	ALTER TABLE users MODIFY COLUMN birth_date DATE	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

Verificamos como quedó estructurada la tabla users, tras este cambio:

```
242 • DESCRIBE transaction_db.users;
```

```
243
```

```
244
```

Result Grid

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(25)	YES		NULL	
surname	varchar(25)	YES		NULL	
phone	varchar(25)	YES		NULL	
email	varchar(50)	YES		NULL	
birth_date	date	YES		NULL	
country	varchar(25)	YES		NULL	
city	varchar(25)	YES		NULL	
postal_code	varchar(15)	YES		NULL	
address	varchar(50)	YES		NULL	
signup_date	date	YES		NULL	
user_segmen...	varchar(25)	YES		NULL	
income_band	varchar(15)	YES		NULL	
region	varchar(15)	YES		NULL	

Result 24

Output

Action Output

#	Time	Action	Message
1	11:16:53	DESCRIBE transaction_db.users	14 row(s) returned

Una vez ya tengo la tabla users lista para trabajar, decidí eliminar las tablas american_users y european_users, considerando que ya tenemos toda su data condensada en esta nueva tabla y no las necesitaremos más, además de que esto sería necesario para cumplir con el requerimiento de modelo de estrella.

```
246 -- Eliminamos las tablas american_users y european_users, ya no las vamos a necesitar
```

```
247 • DROP TABLE american_users;
```

```
248 • DROP TABLE european_users;
```

```
249
```

Output

Action Output

#	Time	Action	Message
2	13:26:23	ALTER TABLE users MODIFY COLUMN birth_date DATE	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
3	13:28:22	DROP TABLE american_users	0 row(s) affected
4	13:28:26	DROP TABLE european_users	0 row(s) affected

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Continuamos con la creación de tablas, creamos la tabla companies:

```
246 -- creamos la tabla companies
247 CREATE TABLE IF NOT EXISTS companies(
248     company_id VARCHAR(15) PRIMARY KEY,
249     company_name VARCHAR(50),
250     phone VARCHAR(25),
251     email VARCHAR(50),
252     country VARCHAR(25),
253     website VARCHAR(50),
254     merchant_category VARCHAR(20),
255     merchant_price_position VARCHAR(20));
256
257
258
259
```

Output

#	Time	Action	Message
1	13:31:26	CREATE TABLE IF NOT EXISTS companies(company_id VARCHAR(15) PRIMARY KEY, company_name VARC...	0 row(s) affected

Cargamos la data para la tabla companies desde el archivo csv:

```
257 -- cargamos la data para companies desde el archivo csv
258 LOAD DATA INFILE
259 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_companies.csv'
260 INTO TABLE companies
261 FIELDS TERMINATED BY ','
262 ENCLOSED BY '"'
263 IGNORE 1 ROWS;
264
265
```

Output

#	Time	Action	Message
1	13:39:18	CREATE TABLE IF NOT EXISTS companies(company_id VARCHAR(15) PRIMARY KEY, company_name VARC...	0 row(s) affected
2	13:39:26	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_companies.csv' INTO TAB...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

Creamos la tabla credit_cards:

```
267 -- creamos la tabla credit_cards
268 CREATE TABLE IF NOT EXISTS credit_cards(
269     id VARCHAR (15) PRIMARY KEY,
270     user_id INT,
271     iban VARCHAR(40),
272     pan VARCHAR(25),
273     pin VARCHAR(10),
274     cvv VARCHAR(5),
275     track1 VARCHAR(100),
276     track2 VARCHAR(100),
277     expiring_date VARCHAR(25),
278     card_type VARCHAR(15),
279     card_renewal_flag BOOLEAN);
280
281
```

Output

#	Time	Action	Message
1	13:41:27	CREATE TABLE IF NOT EXISTS credit_cards(id VARCHAR (15) PRIMARY KEY, user_id INT, iban VARCHAR(4...	0 row(s) affected

Cargamos la data para la tabla credit_cards desde el archivo csv:

```
281 -- cargamos la data para credit_cards desde el archivo csv
282 LOAD DATA INFILE
283 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_credit_cards.csv'
284 INTO TABLE credit_cards
285 FIELDS TERMINATED BY ','
286 ENCLOSED BY '"'
287 IGNORE 1 ROWS;
288
289
```

Output

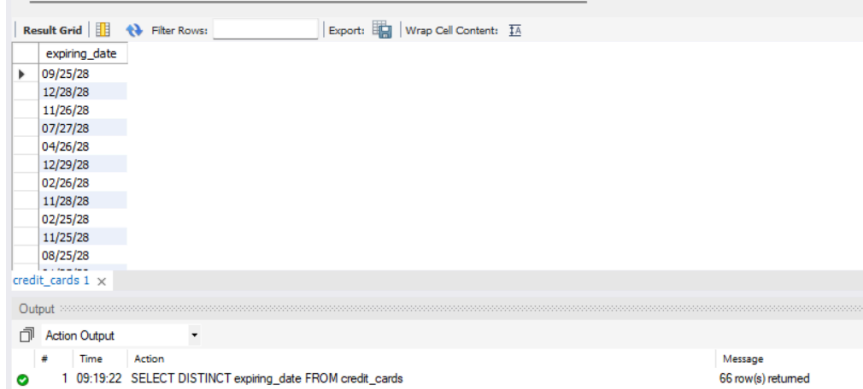
#	Time	Action	Message
1	13:41:27	CREATE TABLE IF NOT EXISTS credit_cards(id VARCHAR (15) PRIMARY KEY, user_id INT, iban VARCHAR(4...	0 row(s) affected
2	13:43:38	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_credit_cards.csv' INTO TA...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

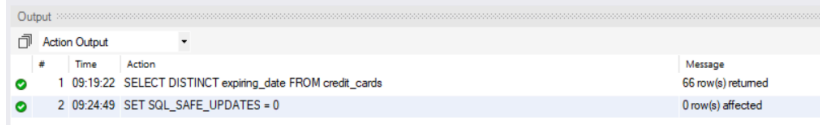
Queremos modificar el tipo de dato de `expiring_date` de `VARCHAR` a `DATE` para que este dato sea más útil en lo sucesivo y nos permita filtrar, ordenar, etc. Primero verificamos que todas las filas tengan el mismo patrón en el dato (repetimos el proceso que hicimos con la columna `'birth_date'` de la tabla `'users'`).

```
288 -- queremos modificar el tipo de dato de expiring_date de VARCHAR a DATE para que este dato sea mas util
289 -- primero verificamos que todas las filas tengan el mismo patron en el dato
290 • SELECT DISTINCT expiring_date FROM credit_cards;
291
```



Desactivamos el modo seguro de `MYSQL` ya que afectamos todas las filas con la modificación.

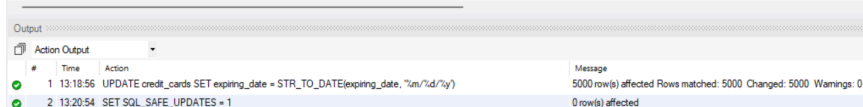
```
292 -- desactivamos el modo seguro SQL ya que vamos a afectar todas las filas
293 • SET SQL_SAFE_UPDATES = 0;
294
```



Modificamos el formato con un string, atendiendo específicamente a la estructura actual de los registros de la columna `expiring_date`: mes/día/año (año con solo dos cifras).

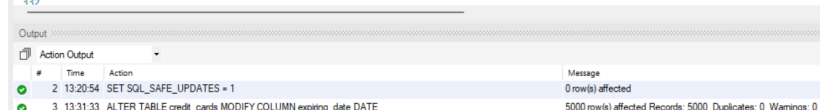
Reactivamos el modo seguro por buenas practicas SQL

```
324 -- Reactivamos el modo seguro por buenas prácticas
325 • SET SQL_SAFE_UPDATES = 1;
326
```



Finalmente, cambiamos el tipo de dato de la columna `expiring_date` de `VARCHAR` a `DATE`

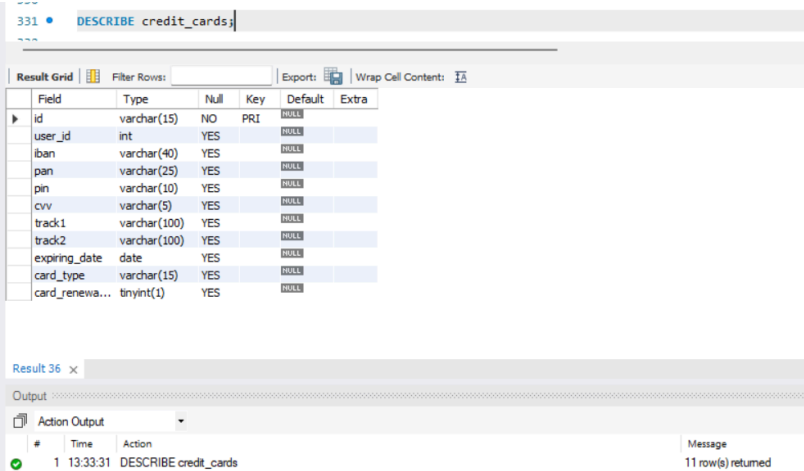
```
327 -- Ahora si cambiamos el tipo de dato de VARCHAR a DATE
328 • ALTER TABLE credit_cards
329   MODIFY COLUMN expiring_date DATE;
330
331
332
```



Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Validamos la descripción de la tabla credit_cards, con atención al tipo de dato de la columna afectada:

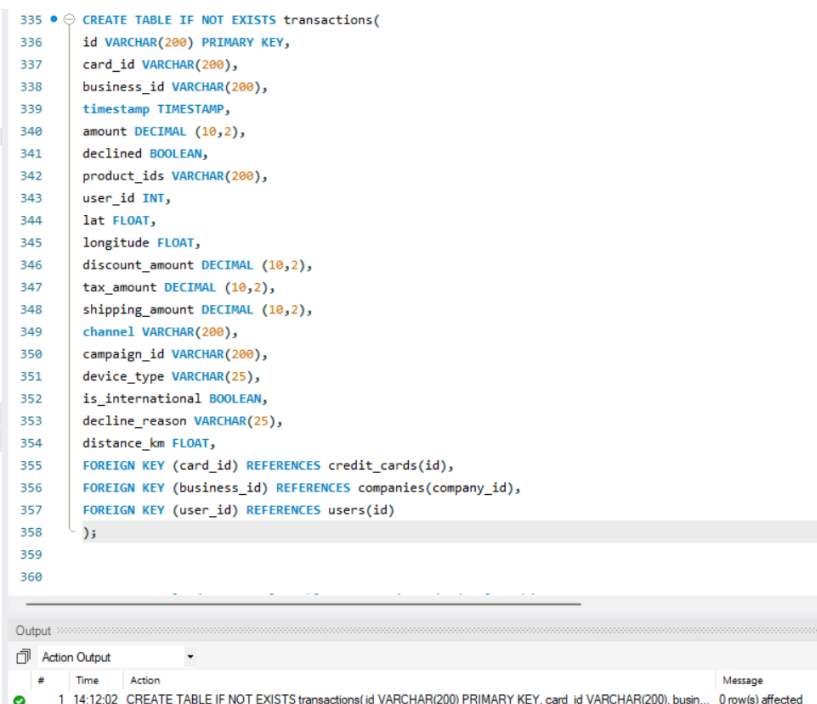


Field	Type	Null	Key	Default	Extra
id	varchar(15)	NO	PRI	NULL	
user_id	int	YES		NULL	
iban	varchar(40)	YES		NULL	
pan	varchar(25)	YES		NULL	
pin	varchar(10)	YES		NULL	
cvv	varchar(5)	YES		NULL	
track1	varchar(100)	YES		NULL	
track2	varchar(100)	YES		NULL	
expiring_date	date	YES		NULL	
card_type	varchar(15)	YES		NULL	
card_renewa...	tinyint(1)	YES		NULL	

En la tabla credit_cards tenemos una columna llamada user_id que se corresponde con el id de la tablas users, sin embargo la relación no fue establecida en la creación de estas tablas, por lo que entendemos, seguimos respetando el modelo de estrella, ya que las tablas de dimensiones se relacionan únicamente a través de la tabla de hechos.

Paso 4. Crear la tabla de hechos, determinando PK y FK, y estableciendo relaciones con las tablas de dimensiones.

Una vez hemos creado y realizado las modificaciones pertinentes en las tablas de dimensiones (tablas padres), procedemos a crear la tabla de hechos (tabla hija), estableciendo de una vez PK y FK y sus relaciones con las demás tablas de la base de datos.



```
335 • CREATE TABLE IF NOT EXISTS transactions(
336   id VARCHAR(200) PRIMARY KEY,
337   card_id VARCHAR(200),
338   business_id VARCHAR(200),
339   timestamp TIMESTAMP,
340   amount DECIMAL (10,2),
341   declined BOOLEAN,
342   product_ids VARCHAR(200),
343   user_id INT,
344   lat FLOAT,
345   longitude FLOAT,
346   discount_amount DECIMAL (10,2),
347   tax_amount DECIMAL (10,2),
348   shipping_amount DECIMAL (10,2),
349   channel VARCHAR(200),
350   campaign_id VARCHAR(200),
351   device_type VARCHAR(25),
352   is_international BOOLEAN,
353   decline_reason VARCHAR(25),
354   distance_km FLOAT,
355   FOREIGN KEY (card_id) REFERENCES credit_cards(id),
356   FOREIGN KEY (business_id) REFERENCES companies(company_id),
357   FOREIGN KEY (user_id) REFERENCES users(id)
358 );
359
360
```

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Cargamos la data para la tabla transactions desde el archivo csv, pero este archivo csv particularmente está separado por ';' y no por ',', por lo que modificamos en función de esta diferencia el código de carga de data.

```
360 -- Cargamos la data para la tabla transactions desde el archivo csv
361 • LOAD DATA INFILE
362 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_transactions.csv'
363 INTO TABLE transactions
364 FIELDS TERMINATED BY ';'
365 ENCLOSED BY ''
366 IGNORE 1 ROWS
367
```

Output

Action Output	Time	Action	Message
1	15:45:24	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_transactions.csv' INTO TA...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0

Visualizamos las tablas existentes en la base de dato 'transaction_db':

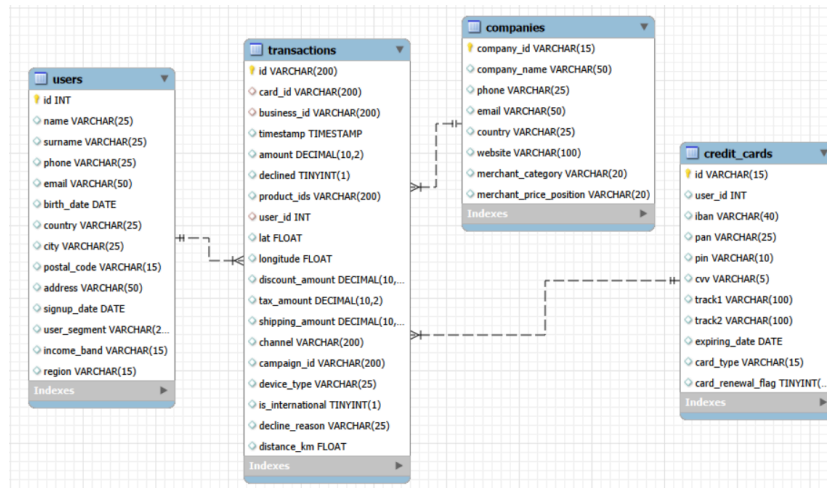
```
367
368 • SHOW TABLES FROM transaction_db
369
```

Result 37 x

Output

Action Output	Time	Action	Message
1	15:45:24	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex-8_transactions.csv' INTO T...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0
2	15:50:14	SHOW TABLES FROM transaction_db	4 row(s) returned

Ejecutamos el proceso de **Reverse Engineering en MySQL Workbench** para comprobar y visualizar el diagrama EER de la base de datos, con un modelo de estrella y relaciones establecidas correctamente entre sus tablas.



Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Ejercicio 9 - Nivel 1

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones usando al menos 2 tablas.

```
371 -- Nivel 1 - Ejercicio 9
372 -- Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones usando al menos 2 tablas.
373
374 • SELECT users.id, CONCAT(users.name, ' ', users.surname) AS nombre_completo
375 FROM users
376 WHERE users.id IN (SELECT transactions.user_id
377 FROM transactions
378 GROUP BY transactions.user_id
379 HAVING COUNT(*) > 80);
380
381
382
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	id	nombre_completo
▶	185	Molly Gilliam
	289	Dxwgi Hwrcu
	318	Bnyr Asbav
	454	Sfzzoh Xgvfricks

Result 45 x

#	Time	Action	Message
1	19:25:23	SELECT users.id, CONCAT(users.name, ' ', users.surname) AS nombre_completo FROM users WHERE users.id IN (SELECT transactions.user_id FROM transactions GROUP BY transactions.user_id HAVING COUNT(*) > 80);	4 row(s) returned

Ejercicio 10 - Nivel 1

Muestra la cantidad media por IBAN para tarjetas de crédito en la empresa Donec Ltd, utiliza al menos 2 tablas.

Nota: este ejercicio lo corregí después de hacer el P2P y enterarme de que si podía usar JOIN, por eso lo uso en este ejercicio y no en los demás, como había mencionado anteriormente.

```
388 • SELECT credit_cards.iban, ROUND(AVG(transactions.amount), 2) AS media_amount
389 FROM credit_cards
390 JOIN transactions
391 ON credit_cards.id = transactions.card_id
392 JOIN companies
393 ON transactions.business_id = companies.company_id
394 WHERE companies.company_name = 'Donec Ltd'
395 GROUP BY credit_cards.iban
396 ORDER BY media_amount;
397
398
399
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	iban	media_amount
▶	US16187838108976709885313	10.26
	US829430002334344253057024	10.62
	US435520063537810061238919	12.54
	CA796226931885124489219768	13.91
	US854598044832250006482562	14.94
	CA344265014403884610135216	15.19
	CA224087292261836747391259	19.13
	US166735578594983908469526	21.68
	CA370114172270364274502939	22.68

Result 17 x

#	Time	Action	Message
1	19:58:34	SELECT credit_cards.iban, ROUND(AVG(transactions.amount), 2) AS media_amount FROM credit_cards JOIN transactions ON credit_cards.id = transactions.card_id JOIN companies ON transactions.business_id = companies.company_id WHERE companies.company_name = 'Donec Ltd' GROUP BY credit_cards.iban ORDER BY media_amount;	371 row(s) returned

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Nivel 2

Ejercicio 1 - Nivel 2

Identifica los cinco días en los que se generó la mayor cantidad de ingresos en la empresa a partir de las ventas. Muestra la fecha de cada transacción junto con el total de ventas.

Respuesta:

En este caso para calcular ingresos, restamos amount(que representa el monto general de la venta) - discount_amount(descuento sobre venta).

```
398 -- Nivel 2 - Ejercicio 1
399 -- 1a. Identifica los cinco días en los que se generó la mayor cantidad de ingresos en la empresa a partir de las ventas.
400 -- Muestra la fecha de cada transacción junto con el total de ventas.
401 • SELECT DATE(transactions.timestamp) AS fecha, SUM(transactions.amount - transactions.discount_amount) AS total_ventas
402 FROM transactions
403 WHERE transactions.declined = 0
404 GROUP BY fecha
405 ORDER BY total_ventas DESC
406 LIMIT 5;
407
408
```

Result Grid		Filter Rows	Export	Wrap Cell Content
fecha	total_ventas			15
2023-02-20	15530.11			
2019-11-18	15244.88			
2019-03-18	14675.70			
2022-12-13	13620.86			
2021-10-27	13277.52			

Result 15 x

Output		
Action Output		
#	Time	Action
1	19:51:31	SELECT DATE(transactions.timestamp) AS fecha, SUM(transactions.amount - transactions.discount_amount) AS ... 5 row(s) returned

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Ejercicio 2 - Nivel 2

Presentar el nombre, número de teléfono, país, fecha y importe, de las empresas que realizaron transacciones con un valor entre 350 y 400 euros y en cualquiera de estas fechas: 29 de abril de 2015, 20 de julio de 2018 y 13 de marzo de 2024. Ordena los resultados de mayor a menor.

```
414 • SELECT DATE(transactions.timestamp) AS fecha, transactions.amount,
415       (SELECT companies.company_name
416        FROM companies
417         WHERE companies.company_id = transactions.business_id) AS nombre_compañia,
418       (SELECT companies.phone
419        FROM companies
420         WHERE companies.company_id = transactions.business_id) AS telefono_compañia,
421       (SELECT companies.country
422        FROM companies
423         WHERE companies.company_id = transactions.business_id) AS pais_compañia
424 FROM transactions
425 WHERE DATE(transactions.timestamp) IN ('2015-04-29', '2018-07-20', '2024-03-13')
426 AND transactions.amount BETWEEN 350 AND 400
427 ORDER BY amount DESC;
```

fecha	amount	nombre_compañia	telefono_compañia	pais_compañia
2015-04-29	390.33	Fringilla LLC	+1 150 784 1716	New Zealand
2024-03-13	383.58	Auctor Mauris Vel LLP	+32 133 447 8239	United States
2015-04-29	369.20	Mauris Institute	+75 473 619 9472	Sweden
2018-07-20	358.64	Pede Cum Ltd	+26 625 798 6126	Norway
2024-03-13	352.21	Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands
2024-03-13	350.71	Tempor Diam Institute	+89 848 141 3136	Netherlands

Result 3 x

Output

Action Output

#	Time	Action	Message
1	15:32:45	SELECT DATE(transactions.timestamp) AS fecha, transactions.amount, (SELECT companies.company_name F...	6 row(s) returned

Ejercicio 3 - Nivel 2

Necesitamos optimizar la asignación de recursos y dependerá de la capacidad operativa que se requiera, así que te piden información sobre el número de transacciones que realizan las empresas, pero el departamento de recursos humanos es exigente y quiere una lista de empresas donde especifiques si tienen igual o más de 400 transacciones o menos.

```
429 -- Nivel 2- Ejercicio 3
430 -- Necesitamos optimizar la asignación de recursos y dependerá de la capacidad operativa que se requiera, así que te piden
431 -- información sobre el número de transacciones que realizan las empresas, pero el departamento de recursos humanos es exigente
432 -- y quiere una lista de empresas donde especifiques si tienen igual o más de 400 transacciones o menos.
433
434 • SELECT COUNT(*) AS numero_transacciones, transactions.business_id,
435       CASE
436         WHEN COUNT(*) >= 400 THEN 'SI'
437         ELSE 'NO'
438       END AS 'Nº Transacciones >= 400'
439 FROM transactions
440 GROUP BY transactions.business_id;
```

numero_transacciones	business_id	Nº Transacciones >= 400
2401	b-2222	SI
410	b-2226	SI
447	b-2230	SI
1514	b-2234	SI
472	b-2238	SI
449	b-2242	SI
1541	b-2246	SI
1511	b-2250	SI
424	b-2254	SI

Result 5 x

Output

Action Output

#	Time	Action	Message
1	16:41:56	SELECT COUNT(*) AS numero_transacciones, transactions.business_id, CASE WHEN COUNT(*) >= 400 THEN ...	100 row(s) returned

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Ejercicio 4 - Nivel 2

Elimina de la tabla de transacciones el registro con ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.

```
442 -- Nivel 2 - Ejercicio 4
443 -- Elimina de la tabla de transacciones el registro con ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.
444 • DELETE FROM transactions
445 WHERE transactions.id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
...
```

Output

#	Time	Action	Message
1	16:58:33	DELETE FROM transactions WHERE transactions.id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected

Ejercicio 5 - Nivel 2

La sección de marketing quiere tener acceso a información específica para llevar a cabo análisis y estrategias efectivas. Se ha solicitado crear una vista que proporcione detalles clave sobre las empresas y sus transacciones.

5a. Tendrás que crear una vista llamada VistaMarketing que contenga la siguiente información: nombre de la empresa. Contacta con el número de teléfono. País de residencia. Compra media realizada por cada empresa.

Respuesta:

En este caso, se nos solicita información de las “compras realizadas”, por lo que nuevamente restamos ‘amount’ - ‘discount_amount’, ya que esto se referiría a lo que realmente compró cada empresa.

```
446 -- Nivel 2 - Ejercicio 5
447 -- La sección de marketing quiere tener acceso a información específica para llevar a cabo análisis y estrategias efectivas.
448 -- Se ha solicitado crear una vista que proporcione detalles clave sobre las empresas y sus transacciones.
449 -- 5a. Tendrás que crear una vista llamada VistaMarketing que contenga la siguiente información: nombre de la empresa.
450 -- Contacta con el número de teléfono. País de residencia. Compra media realizada por cada empresa.
451
452 • CREATE VIEW VistaMarketing AS
453 SELECT companies.company_id, companies.company_name AS nombre_de_la_empresa, companies.phone AS n°_de_telefono,
454 companies.country AS país_de_residencia, (SELECT ROUND(AVG(transactions.amount - transactions.discount_amount))
455 FROM transactions
456 WHERE transactions.declined = 0 AND transactions.business_id = companies.company_id) AS compra_media_realizada
457 FROM companies
458 GROUP BY companies.company_id;
459
460
...
```

Output

#	Time	Action	Message
1	20:08:42	CREATE VIEW VistaMarketing AS SELECT companies.company_id, companies.company_name AS nombre_de_la_empresa, companies.phone AS n°_de_telefono, companies.country AS país_de_residencia, (SELECT ROUND(AVG(transactions.amount - transactions.discount_amount)) FROM transactions WHERE transactions.declined = 0 AND transactions.business_id = companies.company_id) AS compra_media_realizada FROM companies GROUP BY companies.company_id;	0 row(s) affected

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

5b. Presenta la vista creada, ordenando los datos de la compra media más alta a la media más baja.

```
461 -- 5b. Presenta la vista creada, ordenando los datos de la compra media más alta a la media más baja.
462
463 • SELECT *
464 FROM VistaMarketing
465 ORDER BY compra_media_realizada DESC;
```

company_id	nombre_de_la_empresa	nº_de_telefono	pais_de_residencia	compra_media_realizada
b-2222	Ac Fermentum Incorporated	+64 601 244 6320	Germany	316
b-2346	Ut Semper Foundation	+10 283 500 6449	Sweden	294
b-2502	Dolor Vitae Limited	+6 421 567 9195	France	292
b-2574	Orci Adipiscing Limited	+60 897 390 5820	United Kingdom	292
b-2542	Nunc In Foundation	+13 267 741 2619	Italy	292
b-2510	Maecenas Malesuada Fringilla Inc.	+51 673 540 5289	Netherlands	290
b-2370	Non Justo Corp.	+67 963 294 1666	Sweden	290
b-2518	Aliquet Sem Limited	+44 744 425 9915	Netherlands	289
b-2558	Nulla Integer Vulputate Corp.	+75 236 687 9162	Sweden	289

VistaMarketing 18 x

Output

Action Output

#	Time	Action	Message
1	20:09:37	SELECT * FROM VistaMarketing ORDER BY compra_media_realizada DESC	100 row(s) returned

Nivel 3

Ejercicio 1 - Nivel 3

1a. Crea una nueva tabla que refleje el estado de las tarjetas de crédito según si las últimas tres transacciones han sido rechazadas, entonces está inactiva; si al menos una no es rechazada, está activa.

```
470 -- Creamos la nueva tabla credit_cards_status
471 • DROP TABLE IF EXISTS credit_card_status;
472
473 • CREATE TABLE credit_card_status AS
474 SELECT
475     case_transaction.card_id,
476     CASE
477         WHEN SUM(case_transaction.declined) = 3 THEN 'inactive'
478         ELSE 'active'
479     END AS card_status
480 FROM (
481     SELECT
482         transactions.card_id,
483         transactions.declined,
484         ROW_NUMBER() OVER (
485             PARTITION BY transactions.card_id
486             ORDER BY transactions.timestamp DESC
487         ) AS last_transaction
488     FROM transactions
489 ) AS case_transaction
490 WHERE case_transaction.last_transaction <= 3
491 GROUP BY case_transaction.card_id;
```

Output

Action Output

#	Time	Action	Message
1	21:05:21	DROP TABLE IF EXISTS credit_card_status	0 row(s) affected. 1 warning(s): 1051 Unknown table 'transaction_db.credit_card_status'
2	21:05:27	CREATE TABLE credit_card_status AS SELECT case_transaction.card_id, CASE WHEN SUM(case_t...	5000 row(s) affected. Records: 5000 Duplicates: 0 Warnings: 0

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

1b. Basándose en esta tabla, responde: ¿Cuántas cartas hay activas?.

```
493 -- 1b. Basándose en esta tabla, responde: ¿Cuántas cartas hay activas?.
```

```
494
```

```
495 • SELECT COUNT(card_status) AS "active_credit_card"
```

```
496 FROM credit_card_status
```

```
497 WHERE credit_card_status.card_status = "active"
```

```
498 ;
```

```
499
```

```
500
```

```
501
```

active_credit_card
5000

Result 19 x

Output

Action Output

#	Time	Action	Message
2	21:05:27	CREATE TABLE credit_card_status AS SELECT case_transaction.card_id, CASE WHEN SUM(cas...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
3	21:06:33	SELECT COUNT(card_status) AS "active_credit_card" FROM credit_card_status WHERE credit_card_status....	1 row(s) returned

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Ejercicio 2 - Nivel 3

Crea una tabla con la que podamos unir los datos del archivo de products.csv con la base de datos creada (ya que hasta ahora no podíamos hacer esto), teniendo en cuenta que desde la transacción tienes product_ids. Genera la siguiente consulta: Necesitamos saber cuántas veces se ha vendido cada producto.

Respuesta:

Primero vamos a crear la tabla products en nuestra base de datos 'transaction_db':

Cargamos la data a la tabla products desde el archivo csv, reescribiendo en este mismo proceso los datos de las columnas price y cost, para que eliminen el símbolo '\$' que tienen antes del monto y así pueda cargar la data con el tipo de dato que establecimos en la creación de la tabla: DECIMAL.

```
504 -- Creamos una nueva tabla con la información del archivo products
505 • DROP TABLE IF EXISTS products;
506
507 • CREATE TABLE IF NOT EXISTS products(
508     id INT PRIMARY KEY,
509     product_name VARCHAR(50),
510     price DECIMAL (10,2),
511     colour VARCHAR(25),
512     weight DECIMAL(10,2),
513     warehouse_id VARCHAR(25),
514     category VARCHAR(25),
515     brand VARCHAR(25),
516     cost DECIMAL (10,2),
517     launch_date DATE);
518
519 -- Cargamos la data a la tabla products desde el archivo csv
520 • LOAD DATA INFILE
521     'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__products.csv'
522     INTO TABLE products
523     FIELDS TERMINATED BY ','
524     ENCLOSED BY '"'
525     IGNORE 1 ROWS
526     (id, product_name, @price_raw, colour, weight, warehouse_id, category, brand, @cost_raw, launch_date)
527     SET
528         price = REPLACE(@price_raw, '$', ''),
529         cost = REPLACE(@cost_raw, '$', '');
530
531
532
```

Output

#	Time	Action	Message
✓ 1	21:10:15	CREATE TABLE IF NOT EXISTS products(id INT PRIMARY KEY, product_name VARCHAR(50), price DECIMAL...	0 row(s) affected
✓ 2	21:10:29	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__products.csv' INTO TABLE...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Comprobar cómo están almacenados los product_ids en la tabla transactions. Podemos observar que en un mismo registro de transacción, hay uno o más de un product id, en la columna product_ids, esto lo tenemos que resolver más adelante.

```
505 -- Comprobar cómo están almacenados los product_ids en la tabla transactions
506 • SELECT id AS transaction_id, product_ids
507 FROM transactions
508 LIMIT 10;
509
```

Result Grid

transaction_id	product_ids
00043A49-2949-494B-A5DD-A5BAE3B819DD	16, 26, 97, 87
00045D68-ED2E-4F2F-8186-CEE074D875D0	30, 11, 16, 81
000481C3-1C26-4FEF-83A0-ACD0EB004BBD	72
00051AA4-9CBE-4268-B070-C38062A1B3E2	18
0008A312-EDFE-4A4F-8C99-E9C92EC3CA4D	35, 33, 19
0009A151-9BCE-4E31-9053-A468FF77FAAB	93, 55, 28, 91
0009D494-6245-4DF9-955D-2C084191CFFB	55, 8, 72
000A1DEC-CD86-4AB2-A619-71DA88D4A262	46, 56, 73
000A1E64-1414-40B0-9D92-5678A4D958E2	6, 89, 19
000AS879-3472-41D9-AF60-42D3503B543C	58, 61, 55

transactions 2 x

Output

#	Time	Action	Message
1	21:32:05	SELECT COUNT(*) AS total_products FROM products	1 row(s) returned
2	21:33:46	SELECT id AS transaction_id, product_ids FROM transactions LIMIT 10	10 row(s) returned

Ahora, crearemos una tabla intermedia denominada: 'transactions_products, donde tendremos un información de cada transacción por cada uno de los ids de producto.

```
536 -- Creamos una tabla intermedia entre las tablas transactions y products
537 -- que llamaremos transactions_products
538 • DROP TABLE IF EXISTS transactions_products;
539
540 • CREATE TABLE transactions_products (
541     transaction_id VARCHAR(36),
542     product_id INT,
543     PRIMARY KEY (transaction_id, product_id),
544     FOREIGN KEY (transaction_id) REFERENCES transactions(id),
545     FOREIGN KEY (product_id) REFERENCES products(id)
546 );
547
```

Output

#	Time	Action	Message
1	21:13:02	CREATE TABLE transactions_products (transaction_id VARCHAR(36), product_id INT, PRIMARY KEY (tr...	0 row(s) affected

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Vamos a transformar los valores de la columna `product_ids` en la tabla `transactions`, en filas individuales para cada id de producto, mediante `FIND_IN_SET()` y `REPLACE()`.

```
548 -- Transformamos los valores de product_ids en filas individuales mediante FIND_IN_SET() y REPLACE()
549 • SELECT
550     t.id AS transaction_id,
551     p.id AS product_id
552 FROM transactions t
553 CROSS JOIN products p
554 WHERE FIND_IN_SET(
555     p.id,
556     REPLACE(t.product_ids, ' ', '')
557 ) > 0
558 LIMIT 20;
```

transaction_id	product_id
00043A49-2949-494B-A5DD-A58AE38B19DD	97
00043A49-2949-494B-A5DD-A58AE38B19DD	87
00043A49-2949-494B-A5DD-A58AE38B19DD	26
00043A49-2949-494B-A5DD-A58AE38B19DD	16
00045D68-ED2E-4F2F-8186-CEE074D875D0	81
00045D68-ED2E-4F2F-8186-CEE074D875D0	30
00045D68-ED2E-4F2F-8186-CEE074D875D0	16
00045D68-ED2E-4F2F-8186-CEE074D875D0	11
000481C3-1C26-4FEF-83A0-4CDOEB0046BD	72

Result 20 ×

Output

#	Time	Action	Message
1	21:22:29	SELECT t.id AS transaction_id, p.id AS product_id FROM transactions t CROSS JOIN products p WHERE ...	20 row(s) returned

Insertamos los datos de las columnas `transaction_id` y `product_id` en la tabla puente, tomando en cuenta los resultados que obtuvimos tras ejecutar el código anterior.

```
560 -- Insertamos los datos de transaction_id y product_id en la tabla puente
561 • INSERT INTO transactions_products (transaction_id, product_id)
562 SELECT
563     t.id AS transaction_id,
564     p.id AS product_id
565 FROM transactions t
566 CROSS JOIN products p
567 WHERE FIND_IN_SET(
568     p.id,
569     REPLACE(t.product_ids, ' ', '')
570 ) > 0;
```

Output

#	Time	Action	Message
1	21:22:29	SELECT t.id AS transaction_id, p.id AS product_id FROM transactions t CROSS JOIN products p WHERE ...	20 row(s) returned
2	21:23:04	INSERT INTO transactions_products (transaction_id, product_id) SELECT t.id AS transaction_id, p.id AS pr...	253388 row(s) affected Records: 253388 Duplicates: 0 Warnings: 0

Comprobamos cuántos registros hay en nuestra tabla puente: `transactions_products`:

```
566 -- comprobamos cuantos registros hay
567 • SELECT COUNT(*) AS total_rows
568 FROM transactions_products;
```

total_rows
253388

Result 5 ×

Output

#	Time	Action	Message
1	21:59:40	SELECT COUNT(*) AS total_rows FROM transactions_products	1 row(s) returned

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

2b. Genera la siguiente consulta: Necesitamos saber cuántas veces se ha vendido cada producto.

Respondemos la consulta, mostrando cuántas veces se ha vendido cada producto, y diseñamos el código para incluir (en caso de que existan) los productos que no se hayan vendido, es decir, que no estén en la tabla transactions pero si en la tabla products.

Atendiendo a los resultados obtenidos, vemos que tenemos 100 rows, que se corresponden con la cantidad de productos antes comprobados, y que ninguno se vendió menos de 2391 veces (esto lo vemos al ordenarlos de menor a mayor).

```
576 • SELECT
577     products.id AS product_id,
578     (
579         SELECT COUNT(*)
580         FROM transactions_products tp
581         WHERE tp.product_id = products.id
582         AND tp.transaction_id IN (
583             SELECT transactions.id
584             FROM transactions
585             WHERE transactions.declined = 0
586         )
587     ) AS numero_ventas_por_producto
588 FROM products
589 ORDER BY numero_ventas_por_producto ASC;
```

Result Grid

	product_id	numero_ventas_por_producto
▶	43	2391
	42	2404
	19	2423
	22	2428
	71	2437
	69	2437
	75	2438
	77	2439
	37	2440

Result 21 x

Output

#	Time	Action	Message
1	21:24:03	SELECT products.id AS product_id, (SELECT COUNT(*) FROM transactions_products tp	100 row(s) returned

Spring 2. Conceptos básicos de SQL

Realizado por: Lorena Morón

Por último mostramos el diagrama EER de la tabla 'transaction_db', luego de todo el trabajo realizado.

